MISCELLANEOUS TECHNIQUES

1. Weak NFS Privileges (no_root_squash)

Tactics, Techniques, and Procedures (TTP)

- **Tactics:** Privilege Escalation, Defense Evasion.
- **Techniques:** Exploitation of Remote Services, SUID/SGID Executable Creation.
- **Procedures:** Discovering exposed Network File System (NFS) exports, mounting the vulnerable share locally with root capabilities, uploading a custom SUID binary owned by root, and executing it from a low-privileged local shell.

HOW IT WORKS

Network File System (NFS) allows a server to share directories over a network, making them accessible to clients as if they were local storage. When a share is configured on the host machine (defined in `/etc/exports`), the administrator specifies access controls and user mapping options.

The exploitation pipeline relies on the following structured implementation steps:

- **Enumeration:** An attacker runs `showmount -e <target_ip>` to view accessible directories.
- **Configuration Check:** If a directory is exported with the `no_root_squash` directive, any client connecting as their local `root` user retains full administrative privileges over that share on the remote server.
- **Binary Generation:** The attacker compiles a standard C payload designed to spawn a shell with elevated privileges:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>

int main(void){
    setuid(0);
    setgid(0);
    system("/bin/bash");
}
```

- **Mount and Transfer:** The attacker mounts the remote share to their local attack box (`sudo mount -t nfs <target_ip>:/shared_dir /mnt`). They copy the compiled binary into `/mnt`.

- **Setting the SUID Bit:** Because the attacker is `root` on their local system, and `no_root_squash` is active, the file is saved on the target server as owned by `root`. The attacker runs `chmod u+s /mnt/shell`, applying the Set-User-ID (SUID) permission bit.
- **Execution:** The attacker switches to their low-privileged shell on the target system, navigates to the shared directory, and runs `./shell`. The SUID bit forces the operating system to execute the binary under the context of the file owner (`root`), bypassing standard user restrictions.

WHY IT WORKS (THE SECURITY FLAW)

By default, NFS implements **`root_squash`**, a security mechanism that intercepts requests from a remote `root` user and maps (squashes) their identity to the unprivileged `nfsnobody` or `nobody` user account. This prevents remote root users from writing dangerous system files or modifying critical configurations.

When **`no_root_squash`** is enabled, this trust boundary is completely broken. The server inherently trusts the client's assertion of identity. If a user controls their local machine as root, they automatically command root authority over the remote share, allowing them to introduce arbitrary SUID files into the target filesystem.

2. Hijacking Tmux Sessions

Tactics, Techniques, and Procedures (TTP)

- **Tactics:** Privilege Escalation, Lateral Movement.
- **Techniques:** Hijack Execution Flow, Abuse of Valid Sessions.
- **Procedures:** Enumerating running processes for active `tmux` socket servers, checking the Unix socket permissions in the filesystem, and using the `tmux` socket argument (`-S`) to attach directly to a privileged session.

HOW IT WORKS

`tmux` is a terminal multiplexer that allows multiple terminal sessions to be managed from a single window. These sessions remain active in the background even if the user disconnects or closes their primary SSH window.

The mechanism of this attack follows a direct path:

- **Process Discovery:** A low-privileged attacker inspects running processes using command utilities like `ps aux | grep tmux`. They look specifically for sessions initialized with a defined socket path, such as:

```
tmux -S /shareds new -s debugsess
```

- **Permission Auditing:** The attacker inspects the resulting socket file in the filesystem (`ls -la /shareds`). Unix domain sockets behave like standard files regarding access controls.

- **Group Alignment:** The attacker verifies their group memberships via the `id` command to determine if they match the group assigned read/write permissions on the socket (e.g., the `devs` group).
- **Session Attachment:** The attacker executes `tmux -S /shareds` to target the socket file directly. If the session was originally created by the `root` user, the attacker's terminal interface instantly attaches to that active shell, inheriting full administrative capabilities.

WHY IT WORKS (THE SECURITY FLAW)

`tmux` establishes communication between the client interface and the background server process via **Unix Domain Sockets**. These sockets use standard Linux file permissions (Read, Write, Execute for Owner, Group, and Others).

When an administrator intentionally creates a shared session—often to troubleshoot a live issue alongside developers or automated scripts—they modify the socket permissions (`chown root:devs /shareds`) to let other users attach. However, Linux treats permissions globally across the terminal environment. Because the background `tmux` server process itself is running under the context of the user who initiated it (`root`), anyone granted access to write to that socket is given direct, unauthenticated access to run commands inside a root-privileged shell instance.